Hints on web app security

1

- When you develops a web app, you need to protect it

OK, but from WHO and/or from WHAT?

- Vulnerability: a weakness that could allow the execution of actions that are unwanted, usually done with malicious intentions
- At Open Web Application Security Project (OWASP) you can find descriptions of the most common vulnerabilities that you should avoid in your applications

https://owasp.org/

2

Common security vulnerabilities

- Broken authentication / authorization
- Session fixation
- Cross-site scripting (XSS)
- Cross-site request forgery (CSRF)
- Injections
- Sensitive data exposure
- Lack of method access control
- Using dependencies with known vulnerabilities

3

Authentication vs. Authorization

- Authentication: verifying the credentials a user provides with those stored in a system to prove the user is who they say they are. If the credentials match, then you grant access. If not, you deny it

Authentication foils Impersonators

- Authorization: verifying that you're allowed to access an area of an application or perform specific actions, based on certain criteria and conditions put in place by the application

Authorization foils Upgraders

4

(Some) Authentication methods

- There are many types of authentication methods:
  - HTTP Basic: client send a username and password encoded in the Authorization header for each request. The server validates the credentials and grants access if they are correct
  - Form-Based: clients log in using a form by entering their username and password. The server validates credentials, creates a session and stores a session ID (usually in a cookie) to maintain clients' state across requests

5

(Some) Authentication methods

- There are many types of authentication methods:
  - Token-based: clients log in sending credentials to a server that verify them and issues a token. The client stores the token and includes it in the Authorization header for subsequent requests. The server verifies the token's signature and claims to authenticate the clients without tracking session data stored in the server itself.
  - OAuth2: is a framework for authorization. It is designed to allow third-party apps to access resources (e.g. your data) on your behalf, without revealing your credentials.

6

Broken authentication / authorization

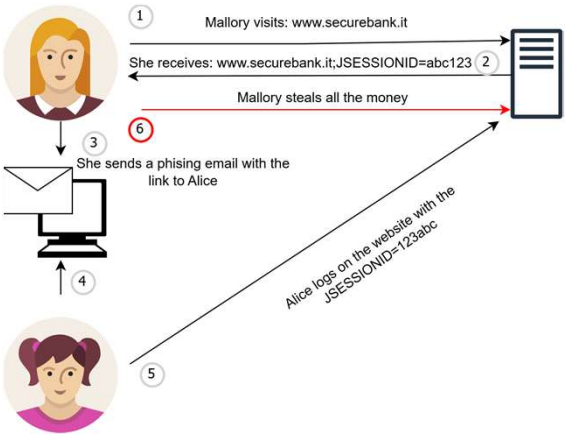
- Broken authentication / authorization occurs when attackers assume other's users identity or acces data for which they do not have access privileges

Pictures from: Head First – Servlets and JSP, 2008

7

Session fixation

- Session fixation occurs when an attacker forces a user to use a specific session ID (one that the attacker already knows) so that after the user logs in, the attacker can hijack the session and act as that user



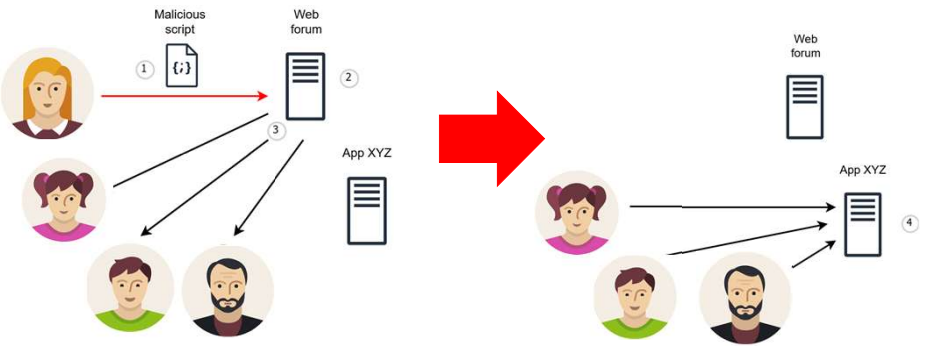
8

Cross-site scripting (XSS)

- XSS occurs when attackers inject client-side scripts into web pages viewed by other users
- Thus, the server sends the attacker's values, usually JavaScript, to visitors, who then run the attacker's code in their own browser. The attacker creates input that the server doesn't clean or sanitize. When a visitor clicks on a link with the attacker's values or accesses a part of the webpage used in the attack, the visitor unknowingly runs the attacker's code

9

Cross-site scripting (XSS): example



1. An attacker adds a comment containing a malicious script
2. The web app does not check the request: it stores and returns it to be displayed
3. Clients executes the script that
4. Will affect App XYZ that will fails or other worse things

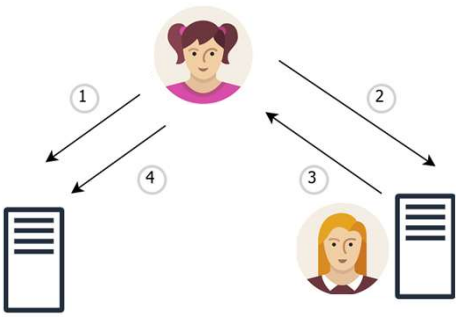
10

Cross-site request forgery (CSRF)

- CSRF occurs when unauthorized commands are submitted from a client that the web application trusts
- It tricks the browser into making a request to a site where the user is already logged in. Because browsers automatically send user's login cookies with every request, the site thinks the user is the one asking for the action

11

Cross-site request forgery (CSRF): example



1. The user log in into their account
2. The same user accesses a page with a script with forgery code
3. The forgery code can execute things on behalf of the user, the script is sent to the user and the browser executes it
4. The script executes actions on behalf of the user

12

Injection

- Injection occurs when attackers introduce specific data into the web app. The purpose is changing data in an unwanted way, or to retrieve data that's not meant to be accessed by the attacker
- Many types of injection attacks exist. Even XSS can be considered one of them. You already knows SQL injection (see Access to DB lectures)

13

Dependencies with known vulnerabilities

- Pay attention to the dependencies you use!
- Sometimes it's not the web app you develop that has vulnerabilities, but the dependencies that you use to build it.
- Eliminate any version that's known to contain a vulnerability. Maven help you!

14

Spring Security

- Spring Security is a framework providing:
  - protection against common vulnerabilities:
    - Authentication / authorization attacks
    - XSS
    - CSRF
    - …
  - session management:
    - Timeout
    - Logout
    - Concurrent session control

15

Adding dependencies

- It integrates easily with Spring Boot and Spring Web

pom.xml

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-
security</artifactId>
</dependency>
```

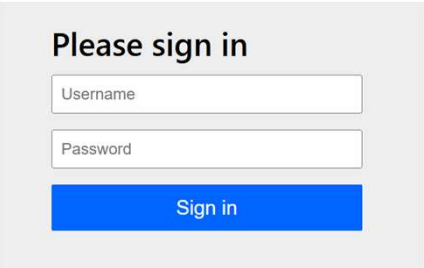
16

What does happens?

17

Example 1 (check esempio7's code)

- Let's consider one of the simplest examples of single-unit web app we coded in a previous lecture
- Add Spring Security dependency and start the web app

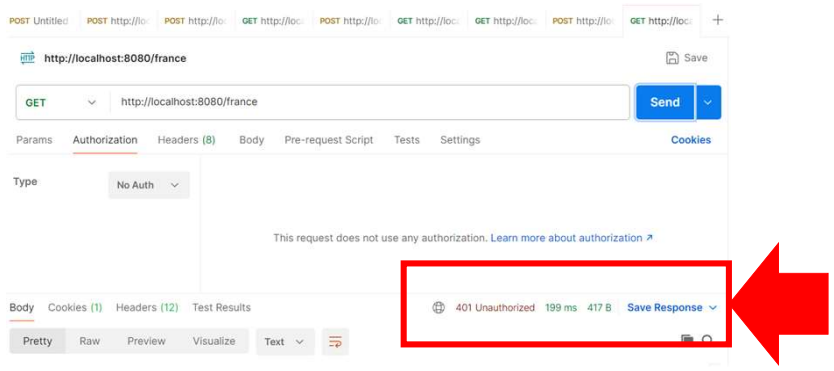


WHO DID IT?
Spring Security!

18

Example 2 (check esempio21's code)

- Let's try now another example with Postman:
  - Start the web app
  - Open Postman and send the request



19

Security starts with filters

- Filters are key components of the Servlet API executed by the servlet container (e.g.Tomcat) that intercept requests / responses before they reach the DispatcherServlet
- They enable cross-cutting concerns such as security and logging

20

Filters in a nutshell

- Filters are used to intercept and process requests before they reach the Dispatcher servlet and responses after they leave the Dispatcher servlet

- Request filters can:
  - perform security checks (e.g. authentication, authorization)
  - reformat request headers or bodies
  - audit or log requests

- Response filters can:
  - compress the response stream
  - append or alter the response stream

21

Going deeper into filters

- Houston, we have a problem…

The Servlet Container knows jakarta.servlet.Filter objects only,
Spring Security lives in Spring and just knows beans

- Thus, we need a bridge between these two worlds:

DelegatingFilterProxy

- It is a true filter that Tomcat can use
- Its only job is to delegate Spring the execution of the filter actions by looking for suitable beans in the Spring Context

22

One or more filters

- The DelgatingFilterProxy delegates Spring through the FilterChainProxy bean which manages one or more SecurityFilterChain beans defining the security rules and filter behavior applied to specific HTTP requests based on URL matching
- Indeed, filters can be chained together to run one after the other
- Filters are designed to be totally self-contained. A filter doesn't care which (if any) filters ran before it did, and it doesn't care which one will run next

23

One or more filters

- Spring Security creates a default SecurityFilterChain
- Such a chain:
  - Secures all the URLs
  - Creates a default user (user) and generates a password in console
  - Enables an automated login form /HTTP Basics authentication
- If you wish you can override the default chain

Who decides in which order the filters will be run?

24

Filters' order

- Spring Security has a list of filters with a prefixed order

REQUEST

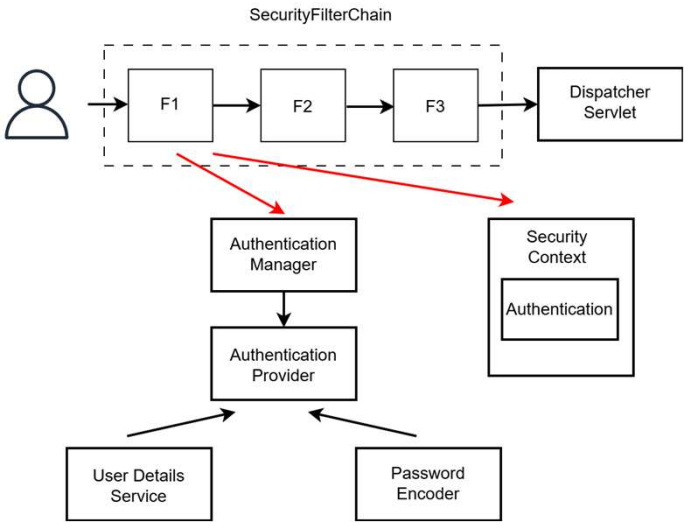
- Infrastructure Filters
- Protocol Security Filters
- Authentication Filters
- Authorization Filters

RESPONSE

- Only if you define your own custom filter, you can choose where it should be inserted

25

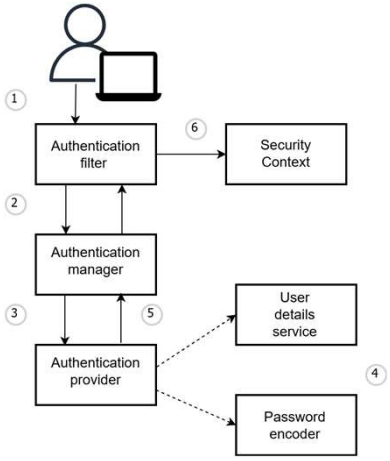
Spring Security's authentication architecture



26

How does it work?

- High-level picture for Spring Security authentication

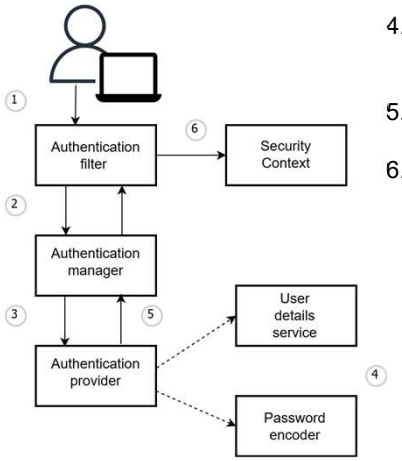


1. The Authentication Filter catches the client request
2. The Authentication Manager takes on the responsabilty for authentication
3. The Authentication Manager employs the Authentication Provider to execute the authentication logic

27

How does it work?

- High-level picture for Spring Security authentication



4. The Authentication Provider finds the user with a User Details Service and validate the password with a Password Encoder
5. The result of the authentication is returned to the Authentication Filter
6. The Security Context keeps the authentication data after the authentication process

All these components have prefigured implementations

28

How does Authentication process work?

- Intercepts the HTTP request
- Extracts credentials (e.g. username and password)
- Crafts an Authentication object (POJO) setting in it:
  - The credentials
  - isAuthenticated() returns false
- Passes the Authentication object to the Authentication Manager to verify it with help of the Authentication Provider
- If the authentication is successful, the Provider creates a new Authentication object setting in it:
  - UsersDetails
  - Roles
  - isAuthenticated() returns true
- Sets that Authentication object in the SecurityContext

29

The Authentication object #1

- Authentication is an interface that, according to the authentication method, can be implemented as:
  - UsernamePasswordAuthenticationToken
  - JwtAuthenticationToken
  - …
- It serves 2 main purposes:
  - Input to Authentication Manager to provide the credentials a user has provided to authenticate
  - Represents the currently authenticated user (see SecurityContext)
- It has several methods:
  - isAuthenticated(): when the filter creates the object, this method is set to return false. It returns true if the authentication process ends

30

The Authentication object #2

- The methods returns different things

Methods	When unauthenticated	When authenticated
getPrincipal()	"anonymousUser" or raw username	UserDetails object
getCredentials()	password / token / credenziali	Null (for security reasons)
getAuthorities()	empty	The roles of the users
getName()	"anonymousUser" or raw username	authenticated username
isAuthenticated()	false	true

31

Authentication Manager

- The Authentication Manager orchestrates the authentication process by delegating the actual verification of credentials to one or more Authentication Provider instances that handle different authentication methods
- Thus, the role of the Authentication Manager is to obtain the list of Authentication Providers configured for the application and to pass the Authentication object to the adequate Authentication Provider

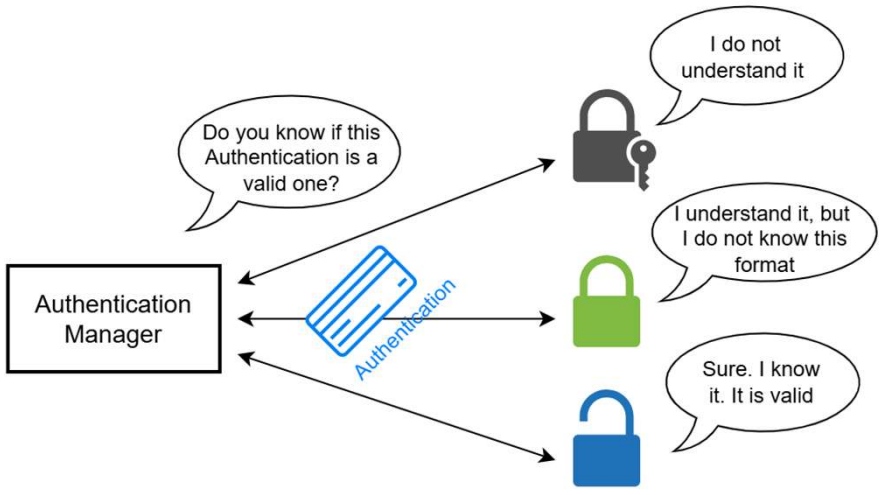
32

Authentication Provider

- The Authentication Provider is the component in which the actual logic for authentication resides. It uses UserDetailsService and PasswordEncoder to retrieve the actual user data and validate it against the provided credentials based on the defined logic
- It has 2 methods:
  - `supports(Class<?> authentication)` that returns true if the current Authentication Provider supports the type provided as an Authentication object
  - `authenticate(Authentication authentication)` that:
    - Throws an exception if the authentication fails
    - Returns a new fully Authentication object that contains all the necessary details about the authenticated entity. Usually, the application also removes sensitive data like a password from this instance. The isAuthenticated() method of this object returns true

33

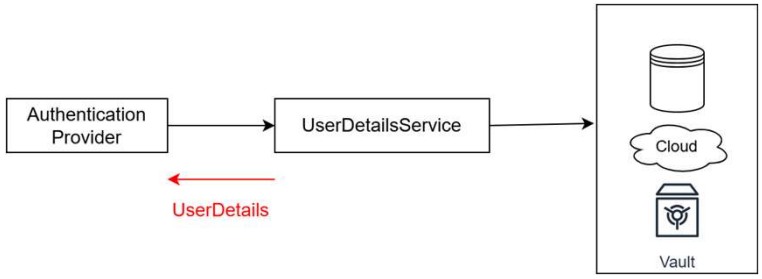
A graphical summary



34

UserDetailsService

- It is an interface that provides a method (i.e., loadUserByUsername) to load the information of a user from a data source (e.g. a DB)
- The Authentication Provider pass the username to the method and it returns a UserDetails object with the user details



35

UserDetailsService

- If you want to do more operations on users (not just loading) you can extend UserDetailsService with UserDetailsManager
- Thus, you can: create, update, remove users, and change passwords (see methods)
- Several implementations exist of UserDetailsManager:

We will use JdbcUserDetailsManager that allows us to manage users in a SQL DB

36

JdbcUserDetailsManager

(Some) Methods	Description
userExists(String username)	Check if a user with that login name exists in the web app
createUser(UserDetails user)	Create a new user with those details
updateUser(UserDetails user)	Update the specified user
deleteUser(String username)	Remove the user with the given login name from the system
changePassword(String oldPassword, String newPassword)	Modify the current user's password

All methods at:
https://docs.spring.io/spring-security/reference/api/java/org/springframework/security/provisioning/JdbcUserDetailsManager.html

37

Pay attention!

- JdbcUserDetailsManager uses 2 tables having as default names Users, Authorities that you must create in your DB
- The table Users must have: `ID,username,password, enabled`
- The table Authorities must have: `ID,username, authority`
- These tables are managed by the JdbcUserDetailsManager, the methods act directly on the tables

38

PasswordEncoder

- It is an interface responsible for hashing passwords and verifying them during authentication
- During authentication, the stored password is not recoverable. The raw password from the request is hashed using the same algorithm, and the resulting hash is compared with the stored one
- Supported hashings:
  - `NoOpPasswordEncoder` ← DO NOT USE IT
  - `StandardPasswordEncoder` ← DO NOT USE IT
  - `Pbkdf2PasswordEncoder`
  - `BCryptPasswordEncoder` ← DEFAULT We will use it
  - `SCryptPasswordEncoder`

39

Pay attention!

- The name PasswordEncoder is misleading!

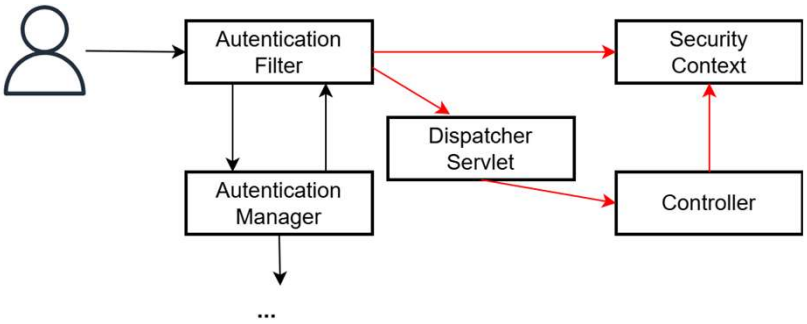
Such interface provide hashing methods
NOT encoding methods!

- Encoding is a reversible transformation used to represent data in a different format, while hashing is a one-way transformation that produces a fixed-size value used to verify data without allowing recovery of the original input

40

SecurityContext

- Once the AuthenticationManager completes the authentication process successfully, the Authentication Filter stores the Authentication object in the SecurityContext



- By default, the SecurityContext is stored in a session (but you can decide to go for a stateless config)

41

How can I configure Spring Security?

- If you decide not to use the default configurations provided by Spring Security, you need to write manually a configuration
- You need to write a class annotated with @Configuration (do you remember it?) in which you build the beans that you need using @Bean annotation
- Tipically, you can configure:
  - UserDetailsManager
  - PasswordEncoder
  - SecurityFilterChain
- Let's have a look at a specific object in the SecurityFilterChain

42